East West University

Department of Computer Science & Engineering

CSE495: IT Project Management

Project Report

BrandON

Digital Advertising Agency Platform

Project Team

Name	Role
Mahir Faysal Haque Dipto	Lead Developer / Project Manager
Mahamudur Rahaman Maharaj	Backend Developer
Nusrat Jahan Oishi	Frontend Developer
Mahamudul Hasan Hamim	QA Engineer / Tester
Israt Jahan Sarker	UI/UX Designer / Documentation

Semester: Spring 2025 | Date of Submission: May 2025

Contents

Executive Summary **3**

1 Initiating **4**

1.1 Project Charter 4

1.1.1 Project Overview 4

1.1.2 Objective 4

1.1.3 Project Scope 5

1.1.4 Stakeholders 5

1.2 PESTEL Analysis 6

1.3 Porter's Five Forces 7

1.4 Business Model Canvas 7

2 Planning **9**

2.1 Work Breakdown Structure (WBS) 9

2.2 Project Schedule 10

2.3 Cost and Effort Estimation 11

2.3.1 Budget Breakdown 12

2.3.2 Software Size Estimation 12

2.3.3 COCOMO Effort and Duration Estimate 13

2.4 Product Backlog 13

2.5 Sprint Backlog 15

2.6 Risk Register 17

3 Executing **18**

3.1 Development Strategy 18

3.1.1 Technology Decisions 18

3.1.2 Team Roles and Work Distribution 19

3.1.3 Handling the Learning Curve 19

3.2 Design Patterns in Execution 19

3.2.1 Singleton Pattern: Prisma Client 20

3.2.2 Strategy Pattern: Pricing Engine 20

3.3 Security Implementation 21

3.3.1 Role-Based Access Control (RBAC) 21

3.3.2 Server-Side Price Validation 22

3.4 Order Pipeline Implementation 22

4 Monitoring and Controlling **23**
4.1 Progress Tracking 23
4.2 Quality Control 23
4.2.1 Automated Test Suite 23
4.2.2 Order Pipeline Enforcement 24
4.2.3 Server-Side Price Validation 24
4.2.4 Mobile Responsiveness 25
4.3 Change Control 25
4.4 Performance Metrics Summary 26

5 Closing **27**
5.1 Final Deployment 27
5.2 Handover Plan 28
5.3 Lessons Learned 28
5.4 Final Project Summary 29

A Project Glossary **31**

B COCOMO Calculation Detail **32**

Executive Summary

BrandON is a web-based platform that connects customers with local banner and sign-board printing shops. The platform lets customers place orders online, upload design files, choose custom sizes, and receive instant price quotes. Admins manage products, pricing tiers, and an order fulfillment pipeline.

The project was built using a modern stack: Next.js 14, TypeScript, Tailwind CSS, Prisma ORM, and PostgreSQL 15, all containerized with Docker. The team delivered an MVP within a budget of 50,000–60,000 BDT, over roughly 5.6 months of effort.

Key outcomes:

- Fully functional web platform with customer, agent, and admin roles.
- Dynamic pricing with three strategy tiers (Standard, Bulk, Rush).
- Strict four-state order pipeline with the statuses Pending, Confirmed, In Progress, and Completed.
- 133 automated Jest tests passing in 2.381 seconds.
- Docker Compose deployment for consistent environments.

Chapter 1

Initiating

1.1 Project Charter

1.1.1 Project Overview

Table 1.1: Project Charter Summary

Field	Details
Project Name	BrandON: Digital Advertising Agency Platform
Project Type	Software Development (Web Application - MVP)
Course	CSE495: IT Project Management
Institution	East West University
Start Date	January 2025
End Date	May 2025
Budget	50,000–60,000 BDT (MVP)
Project Manager	Mahir Faysal Haque Dipto

1.1.2 Objective

The objective of BrandON is to digitalize the order workflow of local printing shops in Bangladesh. The platform replaces manual, walk-in based ordering with a web interface where customers can:

- Browse available banner and signboard products.
- Configure custom dimensions and quantities.
- Upload design files for printing.
- Receive dynamically calculated prices.
- Track their orders through a defined pipeline.

Admins can manage the product catalog, pricing rules, and order statuses from a protected dashboard.

1.1.3 Project Scope

In Scope:

- Customer registration, login, and profile management.
- Product catalog with dynamic pricing (Standard, Bulk, Rush tiers).
- Order placement with file upload support.
- Admin dashboard for product and order management.
- Role-based access control (RBAC): ADMIN, AGENT, CUSTOMER.
- Four-stage order pipeline with status tracking.
- Integration with RedX and Pathao for delivery.
- Docker-based deployment.

Out of Scope (Post-MVP):

- Mobile application (Android/iOS).
- Online payment gateway integration.
- Real-time delivery tracking.
- Multi-vendor support.
- AI-based design generation.

1.1.4 Stakeholders

Table 1.2: Stakeholder Register

Stakeholder	Type	Interest / Role
Printing Shop Owners	Primary / Client	Main users of the admin panel; benefit from digitalized orders
Customers	Primary / Users	Place and track custom print orders online
Delivery Partners (RedX, Pathao)	External	Handle physical delivery of completed orders
CSE495 Course Faculty	Academic Evaluator	Assess project management process and output
Development Team	Internal	Design, build, test, and deploy the platform

1.2 PESTEL Analysis

A PESTEL analysis identifies the macro-environment factors that affect a local printing business moving to a digital platform.

Table 1.3: PESTEL Analysis for BrandON

Factor	Key Points	Impact on BrandON
Political	Government promotes digitalization (Digital Bangladesh 2041 initiative); ICT sector policy support.	Favorable regulatory environment; no major political barrier to launching a digital service platform.
Economic	Bangladesh's SME sector is growing. Internet economy expanding. Low platform development cost.	A startup budget of 50k–60k BDT is practical for an MVP; local shops still need affordable and predictable pricing.
Social	Increasing smartphone and internet penetration in urban Bangladesh. Younger customers prefer online ordering.	Growing demand for online services; customers expect mobile-friendly UI and fast response.
Technological	Open-source stack reduces cost. Docker lowers deployment complexity. Cloud hosting becoming accessible.	Next.js, PostgreSQL, and Docker allow the team to build a production-grade system with minimal cost.
Environmental	Digital orders reduce paper-based communication. Less physical shop visits reduce transport emissions.	Platform contributes indirectly to reduced paper waste in order processing.
Legal	Data privacy regulations are evolving in Bangladesh. Copyright applies to uploaded design files.	Terms of service must clarify file ownership; basic user data protection must be implemented.

1.3 Porter's Five Forces

Table 1.4: Porter's Five Forces Analysis

Force	Level	Analysis
Threat of New Entrants	Medium	Building a basic e-commerce-style platform is technically achievable. However, winning the trust of printing shop owners and building a delivery partner network takes time and local relationships.
Bargaining Power of Suppliers	Low	Printing shops are the service providers. The platform aggregates them, giving BrandON leverage. Individual shops have limited power when listed alongside competitors.
Bargaining Power of Buyers	Medium	Customers can compare prices across shops. The dynamic pricing feature with Bulk discounts and Rush surcharges helps retain customers through transparent, flexible pricing.
Threat of Substitutes	High	Customers can still walk into local print shops. Social media pages (Facebook, WhatsApp) also serve as informal order channels. BrandON must offer clear convenience advantages.
Industry Rivalry	Low–Medium	Dedicated printing platforms in Bangladesh are limited. The market is fragmented with mostly manual operations. BrandON enters a relatively uncontested digital space for this niche.

1.4 Business Model Canvas

Table 1.5: Business Model Canvas: BrandON

Block	Content
Key Partners	RedX and Pathao (delivery logistics) Printing shop owners (service providers) Cloud infrastructure providers (hosting)

Block	Content
Key Activities	Platform development and maintenance Order pipeline management Onboarding printing shops Customer support
Key Resources	Web platform (Next.js, PostgreSQL, Docker) Development team (5 members) Delivery partner agreements Admin dashboard for shop management
Value Propositions	Customers: order custom prints online 24/7 with instant pricing Shops: digitalize operations and reduce walk-in dependency Both sides: transparent order tracking from placement to delivery
Customer Relationships	Self-service ordering portal Status email or notification updates (planned) Admin-managed support
Channels	Web application (primary) Social media promotion (planned) Word-of-mouth from shop owners
Customer Segments	Individual customers needing banners and signboards Small businesses ordering in bulk Event organizers needing rush delivery
Cost Structure	Platform development and operations: 50,000–60,000 BDT (MVP budget) Hosting, maintenance, and updates included Delivery partner fees are transaction-based and scale with orders
Revenue Streams	Commission per completed order (future) Monthly subscription for shops (future) Rush surcharge revenue sharing

Chapter 2

Planning

2.1 Work Breakdown Structure (WBS)

The WBS breaks the BrandON project into five top-level deliverable areas. Each is further divided into specific work packages.

Table 2.1: Work Breakdown Structure: BrandON

ID	WBS Element	Description / Sub-tasks
1	**Project Management**	Project governance and reporting workstream.
1.1	Initiating	Charter, stakeholder identification, PESTEL analysis, Porter's Five Forces analysis
1.2	Planning	WBS, schedule baseline, cost estimate, risk register, product backlog
1.3	Monitoring	Sprint reviews, status reporting, issue tracking, change control
1.4	Closing	Deployment sign-off, handover, final documentation, presentation
2	**System Design**	Architecture and design deliverables for the platform.
2.1	Architecture Design	Next.js App Router structure, RBAC middleware design, database schema, API boundary definition
2.2	UI/UX Wireframes	Public page layouts, order flow screens, admin dashboard mockups
2.3	Database Design	Prisma schema for Material, Order, GalleryItem, ContactMessage, and admin-related entities
3	**Backend Development**	Server-side implementation and integrations.
3.1	Authentication	Admin password flow, admin_token cookie, RBAC middleware enforcement
3.2	Product API	CRUD endpoints for materials, pricing, and inventory visibility
3.3	Order API	Order creation, status-pipeline transitions, server-side price validation
3.4	File Upload	Design file upload handling and storage integration

ID	WBS Element	Description / Sub-tasks
3.5	Delivery Integration	Delivery workflow planning and third-party courier integration hooks
4	**Frontend Development**	Customer-facing and admin-facing application views.
4.1	Public Pages	Home, services, gallery, about, contact, and order pages
4.2	Customer Order Flow	Banner configuration, file upload, order placement, status visibility
4.3	Admin Dashboard	Material management, order management, gallery and message handling
4.4	Responsive Design	Mobile and tablet layout testing with Tailwind CSS refinements
5	**Quality Assurance**	Verification activities across each sprint.
5.1	Unit Testing	Jest tests for validation helpers and pricing logic
5.2	Integration Testing	API endpoint tests with database-backed scenarios
5.3	System Testing	End-to-end workflow tests across public and admin flows
5.4	Regression Testing	Re-run full test suite after fixes and sprint milestones
5.5	Manual Testing	Mobile responsiveness, usability checks, and visual QA
6	**Deployment**	Release preparation and operational handover.
6.1	Docker Configuration	Dockerfile and Docker Compose setup
6.2	Database Migration	Prisma migration workflow for production schema changes
6.3	Environment Config	`.env` setup for deployment secrets and runtime variables
6.4	Handover	Documentation, admin credentials, deployment guide, final client briefing

2.2 Project Schedule

The project was planned over approximately 20 weeks, divided into 4 sprints plus initial planning and final closing phases.

Table 2.2: High-Level Project Schedule

Phase	Activities	Duration	Milestones
Phase 1	Initiation & Planning	Weeks 1–3	Charter approved, WBS complete
Phase 2	Design & Setup	Weeks 4–6	DB schema finalized, wireframes approved
Phase 3	Sprint 1 (Auth + Products)	Weeks 7–9	Login, RBAC, product CRUD working
Phase 4	Sprint 2 (Orders + Pricing)	Weeks 10–12	Order pipeline, dynamic pricing live
Phase 5	Sprint 3 (Dashboard + UI)	Weeks 13–15	Admin and customer dashboards complete
Phase 6	Sprint 4 (QA + Delivery)	Weeks 16–18	133 tests passing, delivery integration done
Phase 7	Deployment & Closing	Weeks 19–20	Docker deployment, handover complete

2.3 Cost and Effort Estimation

2.3.1 Budget Breakdown

Table 2.3: Project Cost Estimation (MVP)

Cost Item	Min (BDT)	Max (BDT)	Notes
Domain, DNS, and email (1 year)	3,000	5,000	Production domain, DNS setup, and business email
Cloud hosting and monitoring (6 months)	8,000	11,000	App server, database hosting, backups, uptime monitoring
Storage and CDN for uploads	2,500	4,000	Design file storage, delivery acceleration, and backup retention
Security and operations	2,500	4,000	SSL, logging, access control hardening, incident response readiness
Development and QA labor	30,000	32,000	Engineering effort, testing, bug fixing, and release stabilization
Contingency reserve	4,000	4,000	Scope-change and production hotfix buffer
Total	**50,000**	**60,000**	Recalculated to match approved budget range

Note: The team used free and open-source tools where possible. The revised budget range includes infrastructure, operations, and labor allocation for a production-ready MVP.

2.3.2 Software Size Estimation

Software size was estimated using two standard methods: Lines of Code (LOC) counting and Function Point (FP) analysis.

Table 2.4: Software Size Metrics

Metric	Value
Estimated Lines of Code (LOC)	3,020 (3.02 KLOC)
Function Points (FP)	106
Average LOC per FP	≈ 28.49

2.3.3 COCOMO Effort and Duration Estimate

The project was classified as an **Organic** type under the COCOMO I model, representing a small team with a well-understood application domain.

The COCOMO Organic model equations are:

$$E = a \times (\text{KLOC})^b = 2.4 \times (3.02)^{1.05} \approx \mathbf{7.73} \text{ person-months} \tag{2.1}$$

$$D = c \times E^d = 2.5 \times (7.73)^{0.38} \approx \mathbf{5.6} \text{ months} \tag{2.2}$$

$$P = E/D = 7.73/5.6 \approx \mathbf{1.38} \text{ persons (average staffing)} \tag{2.3}$$

Where for Organic mode: $a = 2.4$, $b = 1.05$, $c = 2.5$, $d = 0.38$.

Table 2.5: COCOMO Estimation Summary

Parameter	**Formula / Values**	**Result**
Project Size	3,020 lines of code	3.02 KLOC
Function Points	Counted from features	106 FP
COCOMO Type	Organic (small team)	N/A
Effort (E)	$2.4 \times 3.02^{1.05}$	7.73 person-months
Duration (D)	$2.5 \times 7.73^{0.38}$	5.6 months
Avg. Team Size (P)	E/D	$\approx$ 1.38 persons

The estimated 5.6-month duration aligns well with the actual project timeline of approximately 20 weeks ($\approx$ 5 months), confirming that the COCOMO estimate was accurate for this project scope.

2.4 Product Backlog

Table 2.6: Product Backlog: BrandON

ID	**User Story**	**Priority**	**Est. (hrs)**	**Acceptance Criteria**
PB-01	As a customer, I can register and log in securely.	Critical	8	Auth works; invalid credentials rejected

ID	User Story	Priority	Est. (hrs)	Acceptance Criteria
PB-02	As an admin, I have a separate protected admin panel.	Critical	6	Admin cookie enforced via middleware
PB-03	As a customer, I can browse the product catalog.	High	5	Products listed with images and base price
PB-04	As a customer, I can configure custom size and quantity.	Critical	10	Price updates dynamically based on input
PB-05	As a customer, I get Bulk pricing for qty $\geq$ 5.	High	4	10% discount applies automatically
PB-06	As a customer, I can select Rush delivery with +25% surcharge.	High	4	Surcharge shown clearly before checkout
PB-07	As a customer, I can upload design files with my order.	High	8	Files accepted; stored securely
PB-08	As a customer, I can track my order status.	High	6	Order detail page shows the current status as `PENDING`, `CONFIRMED`, `IN_PROGRESS`, or `COMPLETED`, and the label updates after each valid admin-side status change
PB-09	As an admin, I can manage products and pricing.	Critical	10	CRUD operations work on products
PB-10	As an admin, I can update order statuses.	Critical	8	Pipeline transitions enforced server-side
PB-11	As an admin, I can assign orders to agents.	Medium	6	Agent role can view and update assigned orders
PB-12	Server recalculates price to block client-side tampering.	Critical	5	Tampered price in request is ignored
PB-13	Integration with RedX / Pathao for delivery.	Medium	8	Delivery tracking info attached to order
PB-14	Platform is fully mobile responsive.	High	6	All pages pass responsive tests on mobile breakpoints

ID	User Story	Priority	Est. (hrs)	Acceptance Criteria
PB-15	Jest unit and integration tests for all core modules.	High	12	133 tests pass; 0 failures
PB-16	Docker Compose setup for development and production.	High	6	One command spins up full environment

2.5 Sprint Backlog

The product backlog was distributed across four sprints of approximately two to three weeks each.

Table 2.7: Sprint 1: Foundation and Authentication (Weeks 7–9)

ID	Task	Assignee	Est. (hrs)
PB-01	Customer registration and login (NextAuth)	Mahir, Maharaj	8
PB-02	Admin panel protected via `admin_token` cookie middleware	Mahir	6
PB-09	Product CRUD API and admin product management UI	Maharaj, Oishi	10
PB-03	Customer-facing product catalog page	Oishi	5
N/A	Prisma DB schema design and initial migration	Mahir, Hamim	6
N/A	Docker development environment setup	Mahir	4
		Sprint 1 Total	**39 hrs**

Table 2.8: Sprint 2: Orders, Pricing, and Security (Weeks 10–12)

ID	Task	Assignee	Est. (hrs)
PB-04	Custom size and quantity configuration with live price preview	Mahir, Oishi	10
PB-05	Bulk pricing strategy (10% off for qty $\geq$ 5)	Maharaj	4
PB-06	Rush pricing strategy (+25% surcharge)	Maharaj	4
PB-07	Design file upload functionality	Mahim	8
PB-12	Server-side price recalculation to block tampering	Mahir	5
PB-10	Order state pipeline (PENDING $\rightarrow$ COMPLETED)	Mahir, Maharaj	8
		Sprint 2 Total	**39 hrs**

Table 2.9: Sprint 3: Dashboards, Agent Role, and Delivery (Weeks 13–15)

ID	Task	Assignee	Est. (hrs)
PB-08	Customer order tracking dashboard	Oishi, Israt	6
PB-11	Agent role: view and update assigned orders	Maharaj, Israt	6
PB-13	RedX and Pathao delivery integration	Mahim, Maharaj	8
PB-14	Mobile responsive layout for all pages	Oishi, Israt	6
N/A	UI polish: Tailwind styling for all dashboards	Israt	5
		Sprint 3 Total	**31 hrs**

Table 2.10: Sprint 4: Testing, QA, and Deployment (Weeks 16–18)

ID	Task	Assignee	Est. (hrs)
PB-15	Write and run 133 Jest tests (Unit, Integration, System, Regression)	Hamim, Mahir	12
PB-16	Docker Compose production configuration	Mahir	6
N/A	Bug fixes identified in system testing	All	8
N/A	Regression test run after all fixes	Hamim	4
N/A	Final documentation and handover package	Israt	6
		Sprint 4 Total	**36 hrs**

2.6 Risk Register

Table 2.11: Project Risk Register

Risk	Prob.	Impact	Mitigation
Team member unfamiliar with Next.js App Router	High	High	Dedicated learning week; pair programming; online docs
Database connection exhaustion in development	Medium	High	Singleton pattern for Prisma client to prevent connection leaks
Client-side price manipulation by users	High	High	Server-side price recalculation on every order submission
Budget overrun	Low	Medium	Use open-source tools; Docker for free self-hosting
Delivery API breaking changes	Medium	Medium	Abstract delivery layer; easy to swap providers
Scope creep (adding post-MVP features)	Medium	Medium	Strict MVP scope definition; deferred features documented

Chapter 3

Executing

3.1 Development Strategy

The team followed an **Agile Scrum** approach with four sprints, biweekly standups, and a shared Git repository for version control. All five members contributed to both technical and non-technical deliverables.

3.1.1 Technology Decisions

Table 3.1: Technology Stack and Rationale

Technology	Version	Reason
Next.js	14 (App Router)	Full-stack React framework with API routes; replaces separate backend
TypeScript	5.x	Type safety reduces runtime errors; better IDE support
Tailwind CSS	3.x	Utility-first CSS; fast UI iteration without writing custom stylesheets
Prisma ORM	5.x	Type-safe DB access; auto-generates TypeScript types from schema
PostgreSQL	15	Reliable relational DB; handles RBAC and order data well
Docker	24.x	Reproducible environments; same setup for dev, test, and production
Jest	29.x	Industry-standard testing framework for TypeScript/Node.js
NextAuth.js	4.x	Auth solution built for Next.js; handles sessions and providers

3.1.2 Team Roles and Work Distribution

Table 3.2: Team Member Contributions

Member	Primary Role	Key Contributions
Mahir Faysal Haque Dipto	Lead Dev / PM	Architecture, Auth system, RBAC middleware, order API, Docker setup, sprint planning
Mahamudur Rahaman Maharaj	Backend Developer	Pricing strategy implementation, product API, order pipeline state machine, delivery integration
Nusrat Jahan Oishi	Frontend Developer	Customer-facing pages, order placement UI, live price preview, Tailwind styling
Mahamudul Hasan Hamim	QA / Tester	Jest test suite (133 tests), file upload testing, regression testing after each sprint
Israt Jahan Sarker	UI/UX / Docs	Wireframes, admin and agent dashboards, mobile responsiveness, project documentation

3.1.3 Handling the Learning Curve

Two members had limited prior experience with Next.js 14's App Router (introduced in Next.js 13+) and Docker. The team addressed this with:

- **Week 1–2 exploration:** Mahir built a minimal proof-of-concept covering routing, server components, and API routes before the full team began coding.
- **Pair programming:** Oishi (frontend) worked alongside Mahir during the first sprint to understand the App Router structure.
- **Docker-first setup:** Mahir created the `docker-compose.yml` and `Dockerfile` early, so all team members ran the same PostgreSQL instance locally from day one, eliminating "works on my machine" issues.
- **Shared documentation:** The team maintained a short internal wiki of commands, environment variables, and Prisma migration steps.

3.2 Design Patterns in Execution

Two software design patterns were deliberately applied during development to solve specific technical problems.

3.2.1 Singleton Pattern: Prisma Client

Problem: Next.js in development mode uses *hot module reloading* (HMR). Each file change causes Node.js to re-evaluate modules. Without control, that behavior creates a new `PrismaClient` instance on every reload and opens another database connection. PostgreSQL has a limited connection pool, so repeated reloads eventually exhaust the pool and crash the server.

Solution: The Singleton pattern ensures only one `PrismaClient` instance exists per process.

```
// lib/prisma.ts
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
};

export const prisma =
  globalForPrisma.prisma ?? new PrismaClient();

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma;
}
```

By attaching the client to `globalThis`, the instance persists across HMR cycles in development. In production, the module is only evaluated once, so the guard has no effect. This pattern directly prevented database connection exhaustion during the development phase.

3.2.2 Strategy Pattern: Pricing Engine

Problem: The platform requires three different pricing calculations depending on context: standard single-item pricing, bulk order discounts, and rush delivery surcharges. Embedding all three as nested conditionals in the order handler would make the code hard to extend and test.

Solution: The Strategy pattern separates each pricing algorithm behind a common interface.

```
interface PricingStrategy {
  calculate(basePrice: number, quantity: number): number;
}
```

```
class StandardPricing implements PricingStrategy {
  calculate(basePrice: number, quantity: number): number {
    return basePrice * quantity;
  }
}

class BulkPricing implements PricingStrategy {
  // 10% discount for quantity >= 5
  calculate(basePrice: number, quantity: number): number {
    const discount = quantity >= 5 ? 0.90 : 1.0;
    return basePrice * quantity * discount;
  }
}

class RushPricing implements PricingStrategy {
  // +25% surcharge for rush orders
  calculate(basePrice: number, quantity: number): number {
    return basePrice * quantity * 1.25;
  }
}

class PricingContext {
  constructor(private strategy: PricingStrategy) {}
  getPrice(basePrice: number, quantity: number): number {
    return this.strategy.calculate(basePrice, quantity);
  }
}
```

The server selects the appropriate strategy at order submission time based on order type. This design also makes unit testing straightforward: each strategy is tested independently, and the Bulk and Rush strategies each have dedicated test cases in the Jest suite.

3.3 Security Implementation

3.3.1 Role-Based Access Control (RBAC)

RBAC is enforced through a single `admin_token` cookie checked in Next.js middleware (`middleware.ts`). The middleware intercepts all requests to protected routes before they

reach any page or API handler.

- **Public routes** (`/`, `/products`): Accessible by all users.
- **Customer routes** (`/dashboard/...`): Require valid NextAuth session.
- **Admin routes** (`/admin/...`): Require both a valid session and the `admin_token` cookie.
- **Agent routes** (`/agent/...`): Require session with AGENT role.

Centralizing this logic in middleware means no individual page or API route needs to repeat the access check.

3.3.2 Server-Side Price Validation

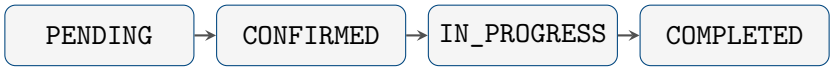
Client-side price displays are for informational purposes only. When an order is submitted, the server ignores the price sent in the request body and recalculates it independently using the same Strategy pattern:

1. Fetch current product price from the database.
2. Identify order type (Standard, Bulk, or Rush) from the order parameters.
3. Apply the correct pricing strategy server-side.
4. Store only the server-calculated price in the order record.

This blocks any attempt to manipulate the price by modifying the HTTP request.

3.4 Order Pipeline Implementation

Orders follow a strict four-state pipeline. Transitions are validated server-side to prevent invalid state jumps.



- **PENDING:** Order placed by customer; awaiting admin review.
- **CONFIRMED:** Admin reviews and accepts the order.
- **IN_PROGRESS:** Printing and production has started.
- **COMPLETED:** Order dispatched to delivery partner (RedX or Pathao).

The API rejects any request that attempts to skip a state or move backwards.

Chapter 4

Monitoring and Controlling

4.1 Progress Tracking

The team used the following mechanisms to track progress throughout the project:

- **Sprint Reviews (biweekly):** At the end of each sprint, the team demonstrated working features. Incomplete items were moved to the next sprint backlog.
- **Git Commit History:** All changes were tracked via Git. Commit messages followed a consistent format, making it easy to trace when each feature was completed.
- **Issue Tracking:** Bugs and new tasks were logged as GitHub Issues and closed when resolved.
- **Burndown Tracking:** The team estimated hours per sprint and tracked remaining work at weekly checkpoints.

4.2 Quality Control

4.2.1 Automated Test Suite

The most important quality control measure was the project's comprehensive Jest testing architecture. Tests were organized into four levels:

Table 4.1: Jest Test Suite Summary

Test Level	What is Tested	Test Count
Unit Tests	Individual functions in isolation: pricing strategies (Standard, Bulk, Rush), utility helpers, input validators.	≈ 45
Integration Tests	API endpoints with a real test database: product CRUD, order creation, authentication flows.	≈ 40
System Tests	Full end-to-end workflows: place order → confirm → complete; admin login and product management.	≈ 30
Regression Tests	Re-run of the full test suite after each sprint to catch newly introduced regressions.	≈ 18
Total	**All tests passing; 0 failures**	**133**

Test Execution Time: The full suite of 133 tests completes in **2.381 seconds**, which allowed the team to run tests frequently without slowing down the development cycle.

The Singleton pattern for the Prisma client was especially important for integration tests: without it, running dozens of tests in parallel would exhaust the database connection pool.

4.2.2 Order Pipeline Enforcement

Quality control extended to the order management logic:

- State transitions are validated server-side. An admin cannot skip CONFIRMED and jump to COMPLETED.
- The API returns a `400 Bad Request` with a clear error message if an invalid transition is attempted.
- Integration tests cover all valid transitions and all invalid transition attempts to confirm rejection.

4.2.3 Server-Side Price Validation

As described in the Executing chapter, the server always recalculates the order price independently. This was tested by:

- Sending order requests with deliberately tampered prices and verifying the stored price matches the server calculation, not the submitted price.

- Testing all three pricing strategies (Standard, Bulk, Rush) with boundary values (e.g., quantity = 4 vs. 5 for Bulk threshold).

4.2.4 Mobile Responsiveness

All customer-facing and dashboard pages were tested for responsiveness at three breakpoints:

- Mobile: 375px (iPhone SE viewport)
- Tablet: 768px (iPad viewport)
- Desktop: 1280px (standard laptop)

Tailwind CSS utility classes (`sm:`, `md:`, `lg:` prefixes) were used to handle layout changes across breakpoints. Pages that failed visual inspection were flagged as bugs and fixed before sprint close.

4.3 Change Control

During the project, one significant scope change was requested: adding a real-time delivery tracking map. This was evaluated against the budget and timeline and deferred to post-MVP. The change was logged, discussed with the team, and formally documented as out of scope.

4.4 Performance Metrics Summary

Table 4.2: Key Quality and Performance Metrics at Project Close

Metric	**Value / Status**
Total automated tests	133
Test failures	0
Full test suite runtime	2.381 seconds
Test coverage areas	Pricing, Auth, Orders, Products, Pipeline
Order pipeline states	4 (PENDING, CONFIRMED, IN_PROGRESS, COMPLETED)
Price tampering prevention	Enforced via server-side recalculation
RBAC enforcement point	Single `middleware.ts` file
Mobile responsiveness	Verified at 375px, 768px, 1280px
Budget status	Within 50,000–60,000 BDT range
COCOMO estimated duration	5.6 months
Actual delivery duration	$\approx$ 5 months

Chapter 5

Closing

5.1 Final Deployment

The production deployment uses Docker Compose to orchestrate two services: the Next.js application and the PostgreSQL 15 database. A single command brings up the entire stack.

```
version: "3.8"
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/brandon
      - NEXTAUTH_SECRET=${NEXTAUTH_SECRET}
      - NEXTAUTH_URL=${NEXTAUTH_URL}
    depends_on:
      - db

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: brandon
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Deployment steps:

1. Copy `.env` file with production secrets (database URL, NextAuth secret, delivery

API keys).

2. Run `docker compose up -d` to start services in the background.
3. Run `npx prisma migrate deploy` to apply database schema migrations.
4. Run `npx prisma db seed` to populate initial product and admin data.
5. Verify at `http://localhost:3000` (or the production domain).

5.2 Handover Plan

The following items were prepared for handover to the client (printing shop owner / operator):

Table 5.1: Handover Checklist

Item	Status	Notes
Source code repository (Git)	Done	Full history; README included
Admin credentials	Done	Delivered securely out-of-band
Deployment guide (step-by-step)	Done	Covers Docker setup and migrations
`.env` template	Done	All required variables documented
User manual (admin panel walkthrough)	Done	Screenshots for each admin function
Delivery partner API keys setup guide	Done	RedX and Pathao configuration steps
Test suite documentation	Done	How to run Jest tests; what they cover
Post-MVP feature list	Done	Documented deferred features for future dev

Training: A one-hour walkthrough session was conducted with the shop owner covering: adding products, managing pricing, processing orders through the pipeline, and viewing completed orders.

5.3 Lessons Learned

- **Early Docker adoption paid off:** Setting up Docker in week one eliminated environment inconsistencies for the entire project. Time invested upfront saved significant debugging time later.

- **Design patterns solve real problems:** The Singleton pattern was not added for architectural elegance. It directly fixed a PostgreSQL connection exhaustion issue discovered during Sprint 1 development. Identifying the root cause and applying the right pattern was more effective than workarounds.
- **Server-side validation is not optional:** Prototyping with client-only price calculations revealed how easily they could be bypassed. Building server-side recalculation early prevented a security gap from reaching production.
- **Tests catch regressions before the user does:** Sprint 4 regression testing caught two pricing-related bugs introduced in Sprint 3 code changes. Without the test suite, these would have reached the client.
- **Scope control protects delivery:** Deferring the real-time delivery map and other post-MVP features kept the team focused. The MVP was delivered on time and within budget as a result.

5.4 Final Project Summary

Table 5.2: Final Project Summary

Item	Summary
Project Name	BrandON: Digital Advertising Agency Platform
Objective	Digitalize custom print order management for local shops
Team Size	5 members
Duration	$\approx$ 5 months (COCOMO estimate: 5.6 months)
Total Effort	7.73 person-months (COCOMO estimate)
Codebase Size	3.02 KLOC / 106 Function Points
Budget Used	Within 50,000–60,000 BDT
Tests Passed	133 / 133 in 2.381 seconds
Design Patterns Used	Singleton (Prisma), Strategy (Pricing)
RBAC Mechanism	`admin_token` cookie, Next.js middleware
Order Pipeline	PENDING $\rightarrow$ CONFIRMED $\rightarrow$ IN_PROGRESS $\rightarrow$ COMPLETED
Delivery Partners	RedX, Pathao
Deployment	Docker Compose
MVP Outcome	Delivered; all critical features functional
Post-MVP Items	Real-time tracking, payment gateway, mobile app

BrandON met its core objectives. The platform delivers a working digital order system for local printing shops, with enforced pricing logic, a secure role-based access model, and a tested order pipeline. The project was completed on time and within budget, using only free and open-source tools.

Appendix A

Project Glossary

Term	Definition
App Router	Next.js 13+ routing system based on the `app/` directory
COCOMO	Constructive Cost Model; estimates software effort from LOC
Docker Compose	Tool to define and run multi-container Docker applications
FP	Function Points; a measure of software functionality
HMR	Hot Module Reloading; Next.js dev mode auto-reload on file change
KLOC	Kilo Lines of Code (1,000 lines)
MVP	Minimum Viable Product
NextAuth	Authentication library built for Next.js
ORM	Object-Relational Mapper; maps code objects to DB tables
Prisma	TypeScript ORM for Node.js and PostgreSQL
RBAC	Role-Based Access Control
Singleton	Design pattern ensuring only one instance of a class exists
Strategy Pattern	Design pattern that selects an algorithm at runtime
WBS	Work Breakdown Structure

Appendix B

COCOMO Calculation Detail

Model type: Organic (small team, well-understood domain)

COCOMO I Organic mode constants:

$$a = 2.4$$
$$b = 1.05$$
$$c = 2.5$$
$$d = 0.38$$

Project size: 3,020 lines of code = 3.02 KLOC

Effort:

$$E = 2.4 \times (3.02)^{1.05} = 2.4 \times 3.221 \approx \mathbf{7.73} \text{ person-months}$$

Duration:

$$D = 2.5 \times (7.73)^{0.38} = 2.5 \times 2.241 \approx \mathbf{5.6} \text{ months}$$

Average staffing:

$$P = \frac{E}{D} = \frac{7.73}{5.6} \approx \mathbf{1.38} \text{ persons}$$

The project was staffed with 5 team members, which is higher than the COCOMO average. This excess staffing enabled parallel work on frontend, backend, QA, and documentation, contributing to on-time delivery within the 5.6-month estimate.